Project 3: SQL Data Analysis

Project Report - what we did, step by step

DecodeLabs Industrial Training Kit | Data Analytics | Batch 2026

Prepared by: Bilal

1. Objective

Use SQL queries to extract business insights from an e-commerce orders dataset. The project asks for SELECT queries, filtering and sorting with WHERE and ORDER BY, grouping with GROUP BY, and the basic aggregations COUNT, SUM and AVG. As the kit suggests, I also experimented with HAVING and with percentage contribution of each category.

Requirement	Where it is covered	Status
SELECT queries	Q1 (preview, KPIs), every query set	Done
WHERE (=, >=, AND, IN, NOT IN, BETWEEN, LIKE, IS NULL)	Q2: 10 filter queries	Done
ORDER BY	Q3, Q4, Q5 (sorted by revenue, date, value)	Done
GROUP BY	Q3 (5 dimensions), Q4 (year, month), Q5	Done
COUNT, SUM, AVG	Q1 KPIs and every GROUP BY result	Done
Extra: HAVING, % contribution, CASE WHEN, LIMIT, NULL handling	Q3, Q5	Done

2. Dataset

The dataset (sheet "Data") contains **1,200 orders** and **14 columns** from Jan 2023 to Jun 2025. It is treated as one SQL table named `orders`. The data was already clean (no duplicate OrderIDs, TotalPrice = Quantity x UnitPrice in every row); the only NULL values are in `CouponCode` (309 rows, meaning no coupon used).

Column	Meaning	Column	Meaning
OrderID	Unique order number	ShippingAddress	Delivery address
Date	Order date	PaymentMethod	5 methods (Credit Card, Online, ...)
CustomerID	Customer code	OrderStatus	Delivered, Shipped, Pending, Cancelled, Returned
Product	7 products (Chair, Desk, ...)	TrackingNumber	Shipment tracking ID
Quantity	Units ordered (1-5)	ItemsInCart	Items in cart (1-10)
UnitPrice	Price per unit	CouponCode	FREESHIP, SAVE10, WINTER15 or NULL
TotalPrice	Quantity x UnitPrice	ReferralSource	5 marketing channels

3. Method - what I did, step by step

1. **Understood the brief.** Read the project kit: SQL is declarative (I describe the result I want, the engine finds the path) and every query follows Input - Process - Output.
2. **Loaded the dataset** into a SQL table (`orders`) and checked structure, data types, NULLs, duplicates and the TotalPrice calculation.
3. **Wrote the SQL queries** in five sets: overview/KPIs, WHERE filters, GROUP BY summaries, time trend, and HAVING / Top-N / conditional aggregation. Each query answers a clear business question.
4. **Thought in execution order** (FROM, WHERE, GROUP BY, HAVING, SELECT, ORDER BY). That is why filters on aggregates use HAVING, not WHERE, and why aliases can be used in ORDER BY but not in WHERE (the "Alias Trap").

5. **Reproduced every result in Excel** with live formulas (COUNTIFS, SUMIFS, AVERAGEIF, SUMPRODUCT) so that each query has a visible, auditable answer on its own sheet.
6. **Verified** all numbers by running the same queries in a real SQL engine (SQLite). Excel and SQL matched on every query (0 formula errors).
7. **Turned results into insights** and charts, including net revenue after removing Cancelled and Returned orders.

4. Queries and results

Q1 - Dataset overview and KPIs (COUNT, SUM, AVG, MIN, MAX)

```
SELECT COUNT(*) AS total_orders, COUNT(DISTINCT CustomerID) AS unique_customers,
       SUM(Quantity) AS units_sold, SUM(TotalPrice) AS total_revenue,
       AVG(TotalPrice) AS avg_order_value, MIN(TotalPrice) AS min_order,
       MAX(TotalPrice) AS max_order
FROM orders;
```

Orders	Customers	Units	Total revenue	Avg order	Min	Max
1,200	1,189	3,535	1,264,761.96	1,053.97	11.39	3,456.40

Q2 - Filtering with WHERE

Ten filters were written; the table shows the condition and the result (COUNT / SUM of TotalPrice).

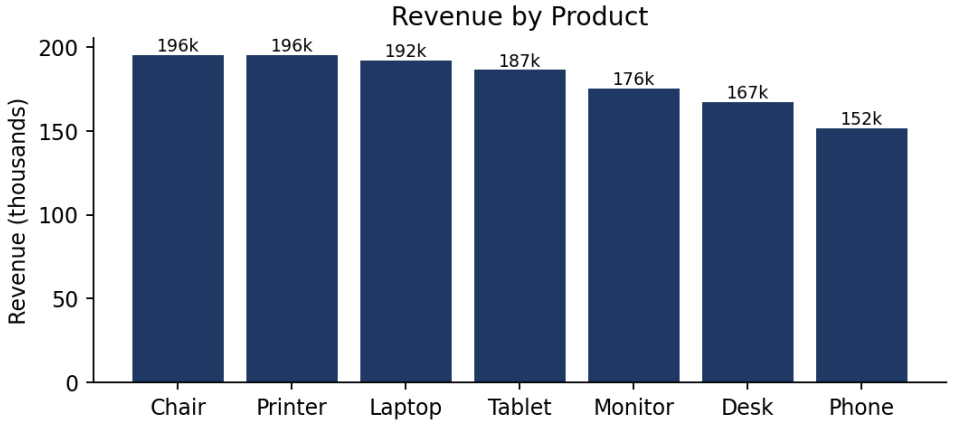
#	WHERE condition	Orders	Revenue
1	OrderStatus = 'Delivered'	231	242,600.32
2	TotalPrice >= 2000	180	461,317.12
3	Product = 'Laptop' AND Quantity >= 3	103	155,714.91
4	OrderStatus IN ('Delivered','Shipped')	466	488,759.90
5	OrderStatus NOT IN ('Cancelled','Returned')	703	745,088.05
6	Date BETWEEN '2024-01-01' AND '2024-12-31'	459	480,235.87
7	ShippingAddress LIKE '5%'	139	156,620.46
8	CouponCode IS NULL	309	322,401.41
9	OrderStatus='Cancelled' AND ItemsInCart BETWEEN 8 AND 10	56	92,335.79
10	PaymentMethod='Gift Card' AND CouponCode='SAVE10'	50	48,007.81

Q3 - GROUP BY with COUNT, SUM, AVG and % contribution

3.1 Revenue by Product

```
SELECT Product, COUNT(*) AS orders, SUM(Quantity) AS units, SUM(TotalPrice) AS revenue,
       AVG(TotalPrice) AS avg_order,
       ROUND(100.0*SUM(TotalPrice)/(SELECT SUM(TotalPrice) FROM orders),1) AS pct
FROM orders GROUP BY Product ORDER BY revenue DESC;
```

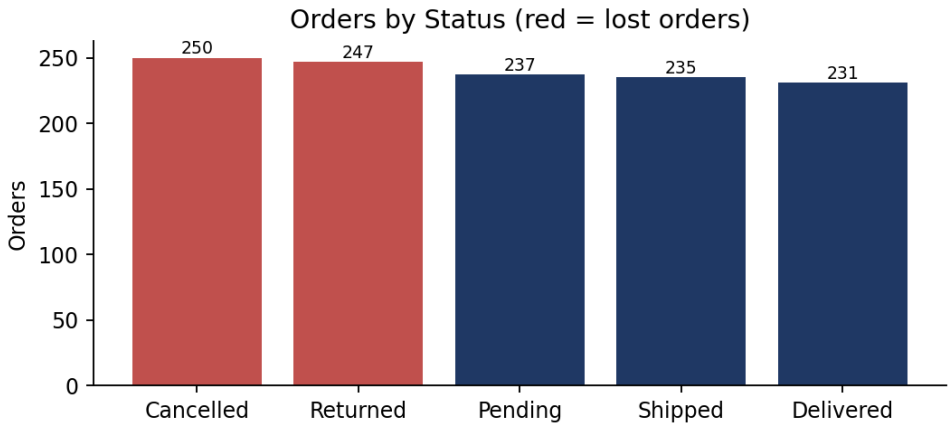
Product	Orders	Units	Revenue	Avg order	% revenue
Chair	178	562	195,620.11	1,098.99	15.5%
Printer	181	542	195,612.61	1,080.73	15.5%
Laptop	173	535	192,126.56	1,110.56	15.2%
Tablet	179	497	186,568.95	1,042.28	14.8%
Monitor	163	480	175,651.41	1,077.62	13.9%
Desk	170	508	167,459.93	985.06	13.2%
Phone	156	411	151,722.39	972.58	12.0%
TOTAL	**1,200**	**3,535**	**1,264,761.96**	**1,053.97**	**100.0%**



3.2 Orders by Order Status

```
SELECT OrderStatus, COUNT(*) AS orders, SUM(Quantity) AS units, SUM(TotalPrice) AS revenue,
       AVG(TotalPrice) AS avg_order,
       ROUND(100.0*SUM(TotalPrice)/(SELECT SUM(TotalPrice) FROM orders),1) AS pct
FROM orders GROUP BY OrderStatus ORDER BY revenue DESC;
```

OrderStatus	Orders	Units	Revenue	Avg order	% revenue
Cancelled	250	759	276,396.21	1,105.58	21.9%
Pending	237	724	256,328.15	1,081.55	20.3%
Shipped	235	680	246,159.58	1,047.49	19.5%
Returned	247	682	243,277.70	984.93	19.2%
Delivered	231	690	242,600.32	1,050.22	19.2%
TOTAL	**1,200**	**3,535**	**1,264,761.96**	**1,053.97**	**100.0%**



3.3 Revenue by Payment Method

```
SELECT PaymentMethod, COUNT(*) AS orders, SUM(Quantity) AS units, SUM(TotalPrice) AS revenue,
       AVG(TotalPrice) AS avg_order,
       ROUND(100.0*SUM(TotalPrice)/(SELECT SUM(TotalPrice) FROM orders),1) AS pct
FROM orders GROUP BY PaymentMethod ORDER BY revenue DESC;
```

PaymentMethod	Orders	Units	Revenue	Avg order	% revenue
Credit Card	234	712	263,847.63	1,127.55	20.9%
Online	258	731	262,442.94	1,017.22	20.8%
Cash	246	753	259,786.29	1,056.04	20.5%
Gift Card	230	675	246,323.92	1,070.97	19.5%
Debit Card	232	664	232,361.18	1,001.56	18.4%
TOTAL	**1,200**	**3,535**	**1,264,761.96**	**1,053.97**	**100.0%**

3.4 Revenue by Referral Source

```
SELECT ReferralSource, COUNT(*) AS orders, SUM(Quantity) AS units, SUM(TotalPrice) AS revenue,
       AVG(TotalPrice) AS avg_order,
       ROUND(100.0*SUM(TotalPrice)/(SELECT SUM(TotalPrice) FROM orders),1) AS pct
FROM orders GROUP BY ReferralSource ORDER BY revenue DESC;
```

ReferralSource	Orders	Units	Revenue	Avg order	% revenue
Instagram	259	770	275,285.45	1,062.88	21.8%
Email	250	746	261,808.55	1,047.23	20.7%
Google	241	714	250,441.48	1,039.18	19.8%
Facebook	228	674	250,410.90	1,098.29	19.8%
Referral	222	631	226,815.58	1,021.69	17.9%
TOTAL	**1,200**	**3,535**	**1,264,761.96**	**1,053.97**	**100.0%**

3.5 Revenue by Coupon Code (NULL kept as its own group)

```
SELECT CouponCode, COUNT(*) AS orders, SUM(Quantity) AS units, SUM(TotalPrice) AS revenue,
       AVG(TotalPrice) AS avg_order,
       ROUND(100.0*SUM(TotalPrice)/(SELECT SUM(TotalPrice) FROM orders),1) AS pct
FROM orders GROUP BY CouponCode ORDER BY revenue DESC;
```

CouponCode	Orders	Units	Revenue	Avg order	% revenue
FREESHIP	313	914	335,036.99	1,070.41	26.5%
nan	309	941	322,401.41	1,043.37	25.5%
SAVE10	286	844	304,840.02	1,065.87	24.1%
WINTER15	292	836	302,483.54	1,035.90	23.9%
TOTAL	**1,200**	**3,535**	**1,264,761.96**	**1,053.97**	**100.0%**

Note: COUNT(*) = 1,200 but COUNT(CouponCode) = 891, because COUNT(column) ignores NULL values.

Q4 - Time trend (GROUP BY year and month)

```
SELECT EXTRACT(YEAR FROM Date) AS yr, COUNT(*) AS orders, SUM(TotalPrice) AS revenue
FROM orders GROUP BY EXTRACT(YEAR FROM Date) ORDER BY yr;
-- (SQLite: strftime('%Y', Date)   MySQL: YEAR(Date)); month query groups by yr, mon
```

Year	Months	Orders	Revenue	Avg / month	Change in avg / month
2023	12	510	552,643.24	46,053.60	n/a
2024	12	459	480,235.87	40,019.66	-13.1%
2025	6	231	231,882.85	38,647.14	-3.4%

2025 contains only Jan-Jun, so averages per month are compared instead of yearly totals.



Q5 - HAVING, Top-N and conditional aggregation

5.1 HAVING: products with revenue above 180,000

```
SELECT Product, SUM(TotalPrice) AS revenue FROM orders
GROUP BY Product HAVING SUM(TotalPrice) > 180000 ORDER BY revenue DESC;
```

Product	Revenue
Chair	195,620.11
Printer	195,612.61
Laptop	192,126.56
Tablet	186,568.95

HAVING filters groups after aggregation; WHERE could not do this because SUM() does not exist yet at the WHERE step.

5.2 HAVING COUNT(*) > 1: repeat customers

```
SELECT CustomerID, COUNT(*) AS orders, SUM(TotalPrice) AS total_spent FROM orders
GROUP BY CustomerID HAVING COUNT(*) > 1 ORDER BY total_spent DESC;
```

CustomerID	Orders	Total spent
C38840	2	5,723.23
C97593	2	2,855.22
C98474	2	2,175.34
C70659	2	1,853.96
C94569	2	1,675.35
C46651	2	1,360.89
C35852	2	1,248.39

CustomerID	Orders	Total spent
C14847	2	1,097.81
C91155	2	704.82
C21191	2	647.77
C56969	2	193.00

5.3 Top 10 orders (ORDER BY ... DESC LIMIT 10)

```
SELECT OrderID, Date, Product, Quantity, OrderStatus, TotalPrice
FROM orders ORDER BY TotalPrice DESC LIMIT 10;
```

OrderID	Date	Product	Qty	Status	TotalPrice
ORD200789	2023-08-17	Tablet	5	Delivered	3,456.40
ORD201122	2023-06-07	Monitor	5	Returned	3,390.95
ORD200632	2023-05-02	Laptop	5	Delivered	3,390.80
ORD200469	2023-11-26	Chair	5	Cancelled	3,384.90
ORD200328	2023-02-28	Tablet	5	Cancelled	3,370.20
ORD200107	2023-03-27	Printer	5	Shipped	3,353.75
ORD200326	2024-07-01	Laptop	5	Returned	3,352.40
ORD201065	2023-10-30	Printer	5	Delivered	3,334.00
ORD201031	2023-02-28	Phone	5	Pending	3,322.55
ORD200463	2023-05-26	Laptop	5	Shipped	3,313.90

5.4 Conditional aggregation (CASE WHEN): lost orders and net revenue by product

```
SELECT Product, COUNT(*) AS orders,
  SUM(CASE WHEN OrderStatus='Cancelled' THEN 1 ELSE 0 END) AS cancelled,
  SUM(CASE WHEN OrderStatus='Returned'  THEN 1 ELSE 0 END) AS returned,
  ROUND(100.0*SUM(CASE WHEN OrderStatus IN ('Cancelled','Returned') THEN 1 ELSE 0 END)/COUNT(*),1) AS loss_rat
  SUM(CASE WHEN OrderStatus NOT IN ('Cancelled','Returned') THEN TotalPrice ELSE 0 END) AS net_revenue
FROM orders GROUP BY Product ORDER BY loss_rate_pct DESC;
```

Product	Orders	Cancelled	Returned	Loss rate	Net revenue
Monitor	163	35	36	43.6%	98,069.37
Tablet	179	34	43	43.0%	103,413.13
Laptop	173	35	39	42.8%	108,710.54
Chair	178	45	28	41.0%	120,320.97
Printer	181	35	38	40.3%	120,998.36
Phone	156	31	31	39.7%	94,534.93
Desk	170	35	32	39.4%	99,040.75
TOTAL	**1,200**	**250**	**247**	**41.4%**	**745,088.05**

5. Key insights

- **Balanced product mix.** Chair (195,620, 15.5%) and Printer (195,613) lead; Phone is last (151,722, 12.0%). No single product dominates.
- **Order fulfilment is the biggest problem.** Only 231 orders (19.2%) are Delivered, while Cancelled + Returned = 497 orders (41.4%). Net revenue is 745,088 instead of the headline 1,264,762.
- **Best channel:** Instagram (21.8% of revenue); weakest: Referral (17.9%).
- **Payments** are evenly spread; Credit Card is first and Debit Card last, only 2.5 percentage points apart.
- **Coupons:** 309 orders (25.8%) used none; FREESHIP is the most used and earns the most revenue among coupons.
- **Declining trend:** average monthly revenue dropped from 46,054 (2023) to 40,020 (2024, -13.1%) and 38,647 in Jan-Jun 2025 (-3.4%).
- **Low loyalty:** only 11 of 1,189 customers ordered twice and nobody ordered more than twice, so repeat purchase is almost absent.
- **Large carts are risky:** 56 cancelled orders had 8-10 items in the cart (total value 92,336), a sign of cart abandonment worth investigating.

Recommendations

- Investigate why 41% of orders are cancelled or returned (stock, delivery time, payment failures) - fixing this is worth more than any new marketing.
- Shift marketing budget towards the strongest channel and test what makes Referral underperform.
- Launch a loyalty or re-order campaign to raise repeat purchases, and review pricing and promotions to reverse the falling monthly revenue.

6. What I learned

- **Declarative thinking:** SQL describes the output; the engine decides how to produce it.
- **Execution order** is FROM, WHERE, GROUP BY, HAVING, SELECT, ORDER BY - not the order we type it.
- **Alias trap:** an alias made in SELECT cannot be used in WHERE (SELECT runs later) but can be used in ORDER BY.
- **WHERE vs HAVING:** WHERE filters rows before grouping; HAVING filters groups after aggregation.
- **NULL behaviour:** COUNT(*) counts every row, COUNT(column), SUM and AVG skip NULLs, so NULLs can skew results if ignored.

7. Deliverables

DecodeLabs_Project3_SQL_Analysis.xlsx - solved workbook: Summary, Data, and one sheet per query set (Q1-Q5) with the SQL text, live-formula results and two charts.
DecodeLabs_Project3_SQL_Report.pdf - this report.
DecodeLabs_Project3_Queries.sql - all queries in one runnable script.

Conclusion: the project covers every requirement (SELECT, WHERE, ORDER BY, GROUP BY, COUNT, SUM, AVG) plus HAVING, CASE WHEN and NULL handling, and turns raw order data into clear, verified business insights.